

Ex no: 6

Date:

A PYTHON PROGRAM TO IMPLEMENT SVM CLASSIFIER MODEL

Aim:

To implement a SVM classifier model using python and determine its accuracy.

Algorithm:

Step 1: Import Necessary Libraries

1. Import numpy as np.
2. Import pandas as pd.
3. Import SVM from sklearn.
4. Import matplotlib.pyplot as plt.
5. Import seaborn as sns.
6. Set the font_scale attribute to 1.2 in seaborn.

Step 2: Load and Display Dataset

1. Read the dataset (muffins.csv) using `pd.read\_csv()`.
2. Display the first five instances using the `head()` function.

Step 3: Plot Initial Data

1. Use the `sns.lmplot()` function.
2. Set the x and y axes to "Sugar" and "Flour".
3. Assign "recipes" to the data parameter.
4. Assign "Type" to the hue parameter.
5. Set the palette to "Set1".
6. Set fit_reg to False.
7. Set scatter_kws to {"s": 70}.
8. Plot the graph.

Step 4: Prepare Data for SVM

1. Extract "Sugar" and "Butter" columns from the recipes dataset and assign to variable ``sugar_butter``.
2. Create a new variable ``type_label``.
3. For each value in the "Type" column, assign 0 if it is "Muffin" and 1 otherwise.

Step 5: Train SVM Model

1. Import the SVC module from the svm library.
2. Create an SVC model with kernel type set to linear.
3. Fit the model using ``sugar_butter`` and ``type_label`` as the parameters.

Step 6: Calculate Decision Boundary

1. Use the ``model.coef_`` function to get the coefficients of the linear model.
2. Assign the coefficients to a list named ``w``.
3. Calculate the slope ``a`` as ``w[0] / w[1]``.
4. Use ``np.linspace()`` to generate values from 5 to 30 and assign to variable ``xx``.
5. Calculate the intercept using the first value of the model intercept and divide by ``w[1]``.
6. Calculate the decision boundary line ``y`` as ``a * xx - (model.intercept_[0] / w[1])``.

Step 7: Calculate Support Vector Boundaries

1. Assign the first support vector to variable ``b``.
2. Calculate ``yy_down`` as ``a * xx + (b[1] - a * b[0])``.
3. Assign the last support vector to variable ``b``.
4. Calculate ``yy_up`` using the same method.

Step 8: Plot Decision Boundary

1. Use the ``sns.lmplot()`` function again with the same parameters as in Step 3.
2. Plot the decision boundary line ``xx`` and ``yy``.

Step 9: Plot Support Vector Boundaries

1. Plot the decision boundary with ``xx``, ``yy_down``, and ``k--``.

2. Plot the support vector boundaries with `'xx'`, `'yy_up'`, and `'k--'`.
3. Scatter plot the first and last support vectors.

Step 10: Import Additional Libraries

1. Import `'confusion_matrix'` from `'sklearn.metrics'`.
2. Import `'classification_report'` from `'sklearn.metrics'`.
3. Import `'train_test_split'` from `'sklearn.model_selection'`.

Step 11: Split Dataset

1. Assign `'x_train'`, `'x_test'`, `'y_train'`, and `'y_test'` using `'train_test_split'`.
2. Set the test size to 0.2.

Step 12: Train New Model

1. Create a new SVC model named `'model1'`.
2. Fit the model using the training data (`'x_train'` and `'y_train'`).

Step 13: Make Predictions

1. Use the `'predict()'` function on `'model1'` with `'x_test'` as the parameter.
2. Assign the predictions to variable `'pred'`.

Step 14: Evaluate Model

1. Display the confusion matrix.
2. Display the classification report.

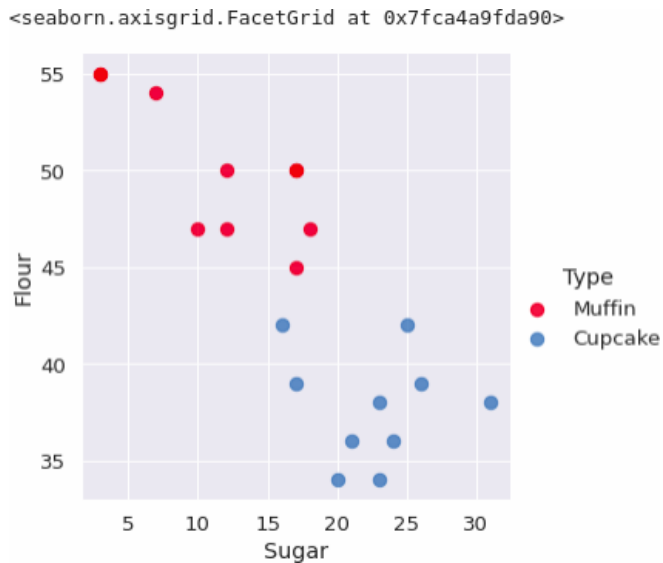
PROGRAM:

```
import numpy as np
import pandas as pd
from sklearn import svm
import matplotlib.pyplot as plt
```

```
import seaborn as sns; sns.set(font_scale=1.2)
recipes=pd.read_csv('../input/muffins-
dataset/recipes_muffins_cupcakes.csv') recipes.head()
recipes.shape
```

```
(20, 9)
```

```
sns.lmplot('Sugar','Flour',data=recipes,hue='Type',palette='Set1',fit_reg=False,scatter_kws={"s":70})
```



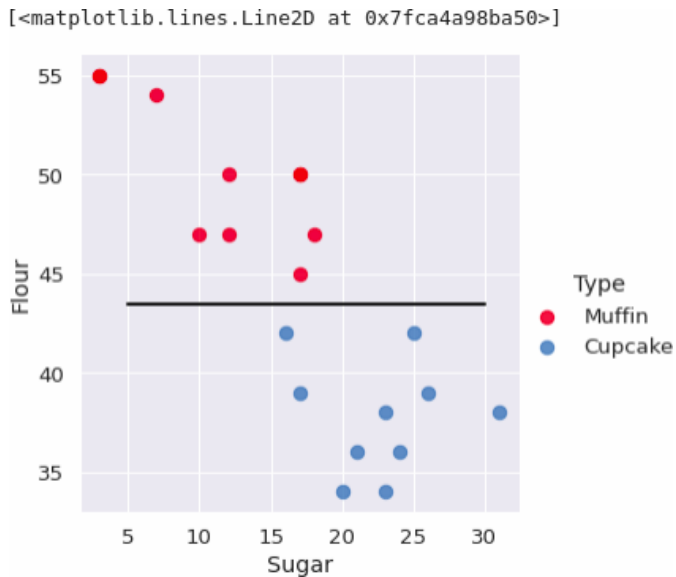
```
sugar_butter=recipes[['Sugar','Flour']].values
type_label=np.where(recipes['Type']=='Muffin',0,1)
model=svm.SVC(kernel='linear')
model.fit(sugar_butter,type_label)
SVC(kernel='linear')
w=model.coef_[0] #seperating the hyperplane
a=-w[0]/w[1]
xx=np.linspace(5,30)
yy=a*xx-(model.intercept_[0]/w[1])
```

```
b=model.support_vectors_[0] #plot to seperate hyperplane that pass
```

```

yy_down=a*xx+(b[1]-a*b[0])
b=model.support_vectors_-[-1]
yy_up=a*xx+(b[1]-a*b[0])
sns.lmplot('Sugar','Flour',data=recipes,hue='Type',palette='Set1',fit_reg=False,scatter_kws={"s":70})
plt.plot(xx,yy,linewidth=2,color='black')

```

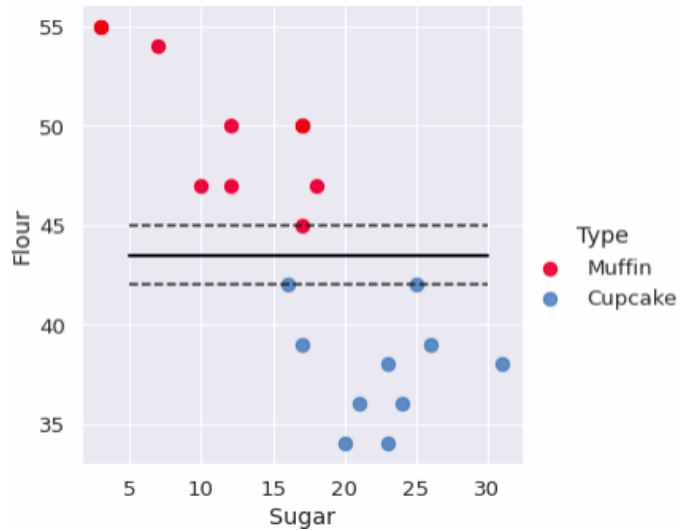


```

scatter_kws={"s":70})
plt.plot(xx,yy,linewidth=2,color='black')
sns.lmplot('Sugar','Flour',data=recipes,hue='Type',palette='Set1',fit_reg=False,scatter_kws={"s":70})
plt.plot(xx,yy,linewidth=2,color='black')
plt.plot(xx,yy_down,'k--')
plt.plot(xx,yy_up,'k--')
plt.scatter(model.support_vectors_[0],model.support_vectors_-[-1],s=80,facecolor='none')

```

<matplotlib.collections.PathCollection at 0x7fca4a88071



```
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
x_train,x_test,y_train,y_test =
train_test_split(sugar_butter,type_label,test_size=0.2)
model1=svm.SVC(kernel='linear')
model1.fit(x_train,y_train)
pred = model1.predict(x_test)
print(pred)

[0 0 1 0]
print(confusion_matrix(y_test,pred))

[[2 0]
 [1 1]]
print(classification_report(y_test,pred))
```